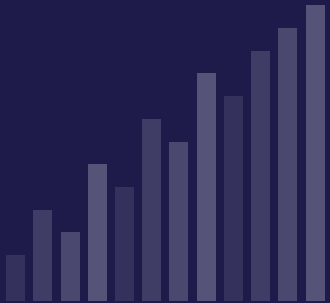
AULA 1 · BLOCO 1: FUNDAMENTOS, ORDENAÇÃO E CUSTO

Algoritmos, eficiência e o método experimental

Cormen et al., *Algoritmos*: capítulo 1



TEORIA

Roteiro da aula

Ao final, você será capaz de

- Definir algoritmo, instância e corretude
- Explicar por que eficiência importa mais que hardware
- Medir tempo de execução em C de forma confiável
- Usar a razão de dobra para reconhecer crescimento linear e quadrático

Teoria

Algoritmos como tecnologia; o exemplo dos dois computadores

Prática em C

Cronômetro em C: relógio monotônico, repetições, melhor de 5

Experimento

Dobrar n e observar o tempo: $\times 2$ ou $\times 4$?

Conexões

IA, criptografia, computação verde e engenharia de desempenho



Teoria

- Cormen et al., *Algoritmos*: capítulo 1
- O que é um algoritmo
- Eficiência vale mais que hardware
- Duas maneiras de medir

TEORIA

O que é um algoritmo

- Um procedimento computacional bem definido que recebe uma **entrada** e produz uma **saída**.
- Resolve um **problema**: a relação desejada entre entrada e saída. Uma **instância** é uma entrada concreta.
- É **correto** se, para toda instância, para com a saída certa.
- Uma **estrutura de dados** organiza os dados para facilitar acesso e modificação: nenhuma serve para tudo.
- Alguns problemas não têm solução eficiente conhecida: os **NP-completos** (caixeiro-viajante, por exemplo).

> Este curso: algoritmos clássicos de ordenação, busca e grafos e as estruturas que os tornam rápidos, sempre com teoria **e** medição.

TEORIA

Eficiência vale mais que hardware

Ordenar $n = 10^7$ números (exemplo da seção 1.2):

	Computador A	**Computador B**
Velocidade	10^{10} instr./s	10^7 instr./s (1000× mais lento)
Algoritmo	inserção, $2n^2$	merge sort, $50\, n \lg n$
Programador	o melhor do mundo, em assembly	mediano, com compilador ruim
Tempo	$\dfrac{2 \cdot (10^7)^2}{10^{10}} = 20\,000$ s $\approx$ **5,5 h**	$\dfrac{50 \cdot 10^7 \lg 10^7}{10^7} \approx 1\,163$ s $<$ **20 min**

O computador 1000 vezes mais lento termina **17 vezes antes**. Com $n = 10^8$, a vantagem passa de 100 vezes.

TEORIA

Duas maneiras de medir

Contar operações
Conta comparações, atribuições ou acessos.

- Independe da máquina e do compilador.
- Exata e reproduzível.
- Ignora cache, desvio de fluxo e memória.

Medir tempo
Relógio antes e depois.

- Mostra o custo real.
- Ruidoso: outros processos, frequência da CPU.
- Pede repetições e o **mínimo** (ou a mediana).

Razão de dobra: se $T(n) \approx c\,n^k$, então $T(2n)/T(n) \approx 2^k$. Razão 2: linear. Razão 4: quadrático. Razão 8: cúbico.



Prática em C

- `clock_gettime(CLOCK_MONOTONIC)`: o relógio certo para medir
- Um algoritmo linear (soma) e um quadrático (pares com soma zero)
- `volatile` impede o compilador de apagar o trabalho

</> PRÁTICA EM C

Armadilhas de quem mede tempo

- **Otimização elimina código morto.** Se o resultado não é usado, `-O2` pode apagar o laço inteiro. Guarde o resultado numa variável `volatile`.
- **Resolução do relógio.** `clock()` mede tempo de CPU em passos grosseiros; use `clock_gettime` e repita até o tempo passar de milissegundos.
- **Ruído do sistema.** Interrupções só aumentam o tempo: o **mínimo** de várias execuções é a estimativa mais limpa.
- **Aquecimento.** A primeira execução paga falhas de página e cache frio.

PRÁTICA EM C

Cronômetro e dois algoritmos (1/3)

```c
#define _POSIX_C_SOURCE 199309L   // clock_gettime com -std=c17
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* Tempo de relógio em segundos (relógio monotônico: não volta com ajustes de hora). */
double agora(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}
```

PRÁTICA EM C

Cronômetro e dois algoritmos (2/3)

```c
long long soma(const int A[], int n) {          // uma passada: cresce como n
    long long s = 0;
    for (int i = 0; i < n; i++) s += A[i];
    return s;
}

long pares_zero(const int A[], int n) {          // todos os pares i < j: cresce como n²
    long cont = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (A[i] + A[j] == 0) cont++;
    return cont;
}
```

PRÁTICA EM C

Cronômetro e dois algoritmos (3/3)

```c
int main(void) {
    int n;
    if (scanf("%d", &n) != 1 || n <= 0) return 1;
    int *A = malloc(n * sizeof(int));
    for (int k = 0; k < n; k++)
        if (scanf("%d", &A[k]) != 1) return 1;
    double t0 = agora();
    printf("soma = %lld\n", soma(A, n));
    printf("pares com soma zero = %ld\n", pares_zero(A, n));
    printf("tempo: %.6f s\n", agora() - t0);
    free(A);
    return 0;
}
```



Experimento

- Hipótese: $T_{\text{lin}} \approx a\,n$ e $T_{\text{quad}} \approx b\,n^2$
- Melhor de 5 execuções, n de 2.000 a 32.000
- Ajuste da constante por mínimos quadrados

EXPERIMENTO

Dobrar n: o tempo dobra ou quadruplica?

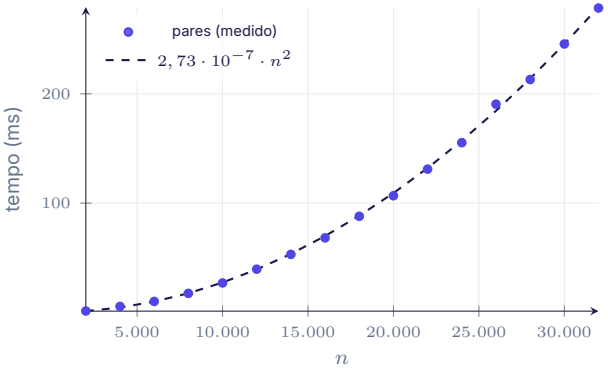
Hipótese. A soma percorre o vetor uma vez: $T(n) \approx a\,n$. A contagem de pares examina $\binom{n}{2}$ pares: $T(n) \approx b\,n^2$. Ao dobrar n, esperamos razões perto de 2 e de 4. As constantes a e b dependem da máquina; as **formas** não.

Método

1. Vetor aleatório de inteiros em $[-1000, 1000]$
2. n de 2.000 a 32.000, passo 2.000
3. Tempo: mínimo de 5 execuções (`CLOCK_MONOTONIC`)
4. Ajuste $c \cdot f(n)$ pelo método dos mínimos quadrados

EXPERIMENTO

Tempo do algoritmo quadrático



n	medido	razão
2.000	1,227	$3,07 \cdot 10^{-7}$
8.000	17,4	$2,71 \cdot 10^{-7}$
14.000	53,1	$2,71 \cdot 10^{-7}$
20.000	106,8	$2,67 \cdot 10^{-7}$
26.000	190,6	$2,82 \cdot 10^{-7}$
32.000	278,6	$2,72 \cdot 10^{-7}$

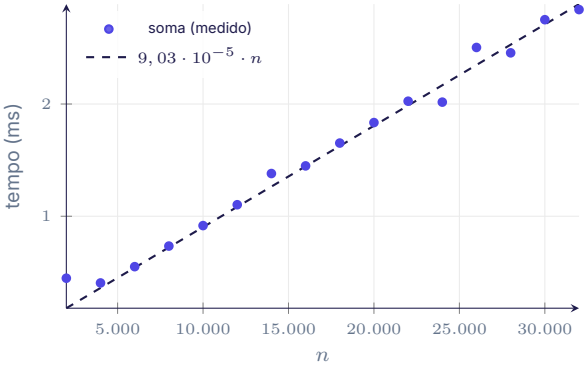
razão = medido $\div\ n^2$

✔ desvio máx. 3,3%

A razão tempo$/n^2$ estabiliza: esse valor é a constante b da sua máquina.

EXPERIMENTO

Tempo do algoritmo linear



n	medido	razão
2.000	0,448	$2,24 \cdot 10^{-4}$
8.000	0,734	$9,18 \cdot 10^{-5}$
14.000	1,381	$9,86 \cdot 10^{-5}$
20.000	1,835	$9,17 \cdot 10^{-5}$
26.000	2,504	$9,63 \cdot 10^{-5}$
32.000	2,841	$8,88 \cdot 10^{-5}$

razão = medido ÷ n

✔ desvio máx. 7,0%

Uma reta. Pontos fora da curva são interrupções do sistema: por isso usamos o mínimo de 5.

EXPERIMENTO

O que o experimento mostrou

Os dois tempos seguem a forma prevista: a curva do quadrático é uma parábola e a do linear, uma reta. Para $n = 32.000$, o algoritmo quadrático já gasta cerca de 100 vezes mais que o linear, e a distância cresce com n. **A forma da curva é uma propriedade do algoritmo; a constante é da máquina.**

Rode na sua máquina: as constantes mudam, as razões de dobra não.

Reproduza: `gcc -O2 aula01_experimento.c -o exp -lm && ./exp`



Conexões

- Inteligência artificial
- Criptografia
- Computação verde
- Engenharia de desempenho

CONEXÕES

Onde isso aparece na computação

Inteligência artificial

Treinar um modelo de linguagem custa da ordem de $6 \times$ parâmetros $\times$ tokens operações: a análise de custo decide quais modelos são viáveis.

Criptografia

A segurança do RSA depende de não existir algoritmo eficiente conhecido para fatorar inteiros grandes: a ineficiência vira proteção.

Computação verde

Data centers consomem cerca de 1–2% da energia elétrica mundial. Trocar n^2 por $n \lg n$ economiza energia, não só tempo.

Engenharia de desempenho

Ferramentas como `perf`, `gprof` e testes de regressão de desempenho em integração contínua aplicam o método desta aula na indústria.

PARA CASA

Para casa

Exercícios

1. (1.1-1) Dê um exemplo do mundo real que exija ordenação e um que exija o menor caminho.
2. (1.2-2) Para que n a inserção ($8n^2$ passos) vence o merge sort ($64\,n \lg n$ passos)?
3. (1.2-3) Qual o menor n em que $100\,n^2$ é mais rápido que 2^n?
4. Rode o experimento com `-O0` e com `-O2`. O que muda: a forma ou a constante?
5. Implemente uma contagem de pares em $O(n \lg n)$ (ordene e use dois índices) e meça.

Leitura
Cormen et al., capítulo 1 e problema 1-1.

Código
josedihego.net/estruturas